

/ AGENTS.md — Cursor Execution Rules for Matrix AllInAll

1. الدور الأساسي

أنت تعمل داخل مشروع Matrix / AllInAll. دورك ليس إعادة بناء المشروع بالكامل، وليس تنفيذ أفكار كبيرة دفعة واحدة. دورك هو تنفيذ تغييرات صغيرة، آمنة، قابلة للمراجعة، مع تحليل الأخطاء قبل تعديل الكود.

يجب أن تتصرف كمطور Senior حذر. لا تفترض. لا تكسر البنية الحالية. لا تعيد كتابة ملفات كبيرة بدون سبب قوي.

2. قاعدة مهمة جدًا

لا تنفذ أي مهمة كبيرة مرة واحدة.

أي طلب كبير يجب تقسيمه إلى مهام صغيرة جدًا:

Analyze → Plan → Change small part → Run checks → Fix errors → Document result

لا تنتقل للمرحلة التالية قبل نجاح المرحلة الحالية.

3. قبل تعديل أي كود

قبل أي تعديل:

1. اقرأ الملف المطلوب تعديله كاملاً.
2. افهم علاقته بباقي المشروع.
3. ابحث عن imports والأنواع المستخدمة.
4. افحص إن كان الملف Server Component أو Client Component.
5. افحص إن كان الكود يؤثر على:

Auth .6

Permissions .7

Database .8

API .9

Public forms .10

Customer data .11

Financial data .12

13. اكتب ملخصًا قصيرًا لما ستعده.

4. لا تضغط على نفسك

لا تحاول حل كل شيء في رد واحد.

لو وجدت أكثر من خطأ:

- أصلح أول خطأ واضح فقط -
- `build/typecheck` شغل -
- ثم انتقل للخطأ التالي -

لا تعمل تغييرات ضخمة لعلاج خطأ صغير.

5. ملف تتبع الأخطاء

يجب إنشاء أو تحديث ملف:

`docs/progress/error-analysis.md`

كل خطأ يتم تسجيله بهذه الصيغة:

```
## Error: short title

### Date
YYYY-MM-DD

### Area
web-marketing / web-platform / api / worker / packages

### Error Message
Paste exact error here.

### Root Cause
Explain the real cause, not guesses.

### Files Affected
- path/to/file.tsx

### Fix Applied
Explain the exact change.
```

Verification

Command run:

```
```bash
```

```
pnpm ...
```

Result: success / failed

### Notes

.Any warning, risk, or follow-up

أو runtime أو build ممنوع إصلاح أخطاء بدون تسجيلها في هذا الملف، إذا كان الخطأ من API أو TypeScript.

---

## أسلوب تحليل الأخطاء. 6.

عند ظهور خطأ:

1. اقرأ رسالة الخطأ كاملة.
2. فقط pnpm لا تركز على آخر سطر.
3. ابحث عن أول سطر يحتوي على:

- Failed to compile
- Type error
- Module not found
- Cannot read properties
- Prisma error
- Database error
- Hydration error

4. حدد:

- الملف.
- السطر.
- السبب الحقيقي.
- runtime أم architecture أم type هل الخطأ.

5. أملح السبب وليس العرض.

مثال:

```
```text
```

```
[ERR_PNPM_RECURSIVE_RUN_FIRST_FAIL]
```

هذا ليس السبب الحقيقي. السبب الحقيقي يكون أعلى منه في اللوح.

7. قواعد TypeScript

- لا تستخدم `any` إلا عند الضرورة ومع تعليق يشرح السبب.
- لا تستخدم `as any` كحل سريع.
- لا تكتم TypeScript errors.
- لا توقف ESLint فقط لتعدي build.
- عرّف الأنواع بوضوح.
- استخدم union types عند الحاجة.
- اجعل الـ DTOs والـ validation متناسقة.
- استخدم Zod أو schemas واضحة للمدخلات.

ممنوع:

```
// ممنوع إلا لسبب قوى
const data: any = ...
```

الأفضل:

```
type LeadInput = {
  name: string;
  phone: string;
  projectType: string;
};
```

8. قواعد الأمان

يجب بناء كل شيء بافتراض أن النظام سيستخدم بيانات عملاء حقيقية.

ممنوع

- تخزين كلمات مرور plain text.
- وضع API keys داخل الكود.
- تسجيل بيانات حساسة في console logs.
- إرسال بيانات عميل كاملة لطرف خارجي بدون ضرورة.
- فتح API endpoints بدون حماية.
- الاعتماد على client-side validation فقط.
- السماح بحذف نهائي بدون صلاحية واضحة.
- كشف أرباح أو بيانات مالية بدون permission.
- خلط بيانات tenants أو العملاء.

مطلوب

- استخدام environment variables للأسرار.
- Hashing لكلمات المرور.
- Server-side validation.
- Rate limiting للـ public forms.
- Audit log للعمليات الحساسة.
- RBAC permissions.
- Tenant isolation.
- Soft delete بدل hard delete.
- Error messages لا تكشف معلومات حساسة.
- CSRF/session protection حسب طريقة auth.

9. التعامل مع البيانات الحساسة

أي بيانات من هذا النوع تعتبر حساسة:

- أرقام الهواتف.
- البريد الإلكتروني.
- بيانات العملاء.
- بيانات مالية.
- مدفوعات.
- صلاحيات.
- Tokens.
- Passwords.
- Session IDs.
- Tenant IDs.
- ملفات مرفوعة من العملاء.

لا تكتب هذه البيانات في logs إلا إذا كانت masked.

مثال:

```
01234567890 → 012****7890  
email@example.com → e***@example.com
```

10. قواعد التشفير والحماية

عند التعامل مع كلمات المرور:

- استخدم bcrypt أو argon2.
- لا تحفظ كلمة السر الأصلية.
- لا ترسل كلمة السر في responses.

عند التعامل مع tokens:

- خزنها في .env.
- لا تطبعها في logs.
- لا تضعها في frontend bundle.
- لا تضعها في Git.

عند التعامل مع ملفات:

- تحقق من نوع الملف.
- تحقق من الحجم.
- لا تسمح بتنفيذ ملفات مرفوعة.
- خزّن metadata فقط في DB.
- استخدم storage آمن لاحقًا.

11. قواعد قاعدة البيانات

قبل تعديل Prisma schema:

1. افهم العلاقات الحالية.
2. لا تحذف models أو fields قائمة بدون migration واضحة.
3. لا تغير اسم field مستخدم في الكود بدون تحديث كامل.
4. أضيف indexes للحقول التي تستخدم في البحث.
5. استخدم enums للحالات.
6. استخدم createdAt و updatedAt.
7. استخدم archivedAt أو deletedAt بدل الحذف النهائي.

كل entity مهم يجب أن يحتوي على:

```
id
createdAt
updatedAt
createdBy إن أمكن
tenantId عند الحاجة
status عند الحاجة
```

12. قواعد API

أي endpoint يجب أن يحتوي على:

1. Validation.
2. Auth check إذا كان داخليًا.
3. Permission check.

- .4. Tenant check
- .5. Error handling
- .6. Response type واضح.
- .7. Audit log إذا كان يغير بيانات حساسة.

ممنوع أن تعتمد على frontend فقط.

13. قواعد الواجهات UI

أي شاشة يجب أن تحتوي على:

- Loading state
- Empty state
- Error state
- Success state
- Responsive layout
- Arabic RTL مضبوط.
- English LTR مضبوط إن وجدت.
- Labels واضحة.
- لا توجد بيانات وهمية في production.

لو تحتاج demo data، ضعها فقط في seed أو dev-only file، وليس داخل production UI.

14. قواعد الفورمات

أي form يجب أن يحتوي على:

- Client validation
- Server validation
- Clear error messages
- Submit loading state
- Disabled submit while sending
- Protection against duplicate submit
- Autosave draft عند الفورمات الطويلة.
- عدم ضياع البيانات عند خطأ الإرسال.
- عدم reset إلا بعد success مؤكد.

ممنوع:

```
event.currentTarget.reset()
```

بعد `await` بدون حفظ مرجع الفورم أولاً.

الأفضل:

```
const form = event.currentTarget;

await submitData();

form.reset();
```

أو:

```
formRef.current?.reset();
```

15. قواعد العمل مع Next.js

- افصل Server Components عن Client Components.
- لا تستخدم browser APIs في Server Components.
- أي component يستخدم `useState` , `useEffect` , `localStorage` , `window` يجب أن يبدأ بـ:

```
'use client';
```

- لا تستخدم `require()` داخل ملفات TypeScript الحديثة إلا للضرورة.
- استخدم `imports`:

```
import path from 'node:path';
import { readFile } from 'node:fs/promises';
```

- بعد أي تعديل مهم شغل `build`.

16. أوامر الفحص المطلوبة

بعد كل تعديل في `web-marketing`:

```
pnpm --filter @allinall/web-marketing build
```

بعد كل تعديل في `web-platform`:

```
pnpm --filter @allinall/web-platform build
```

بعد كل تعديل في API:

```
pnpm --filter @allinall/api build
```

ولو يوجد tests:

```
pnpm test
```

لا تعتبر المهمة انتهت بدون تشغيل الفحص المناسب.

17. ممنوعات صارمة

ممنوع:

- `git add .`
- حذف ملفات كثيرة بدون إذن.
- إعادة كتابة المشروع بالكامل.
- نقل architecture بالكامل بدون خطة.
- كنم أخطاء build.
- وضع secrets في الكود.
- استخدام بيانات وهمية في production.
- تعديل صفحات كثيرة بسبب خطأ في صفحة واحدة.
- تغيير أسماء routes بدون تحديث الروابط.
- إزالة validation لتسهيل build.
- إلغاء ESLint rule بدل إصلاح السبب إلا بعد موافقة.

18. طريقة تنفيذ أي مهمة

اتبع هذا الشكل:

1. Understand the request.
2. Inspect relevant files.
3. Write small plan.
4. Implement smallest safe change.
5. Run checks.
6. Fix only errors caused by this change.
7. Update docs/progress/error-analysis.md if errors were involved.
8. Summarize changed files and verification result.

19. عند عدم التأكد

لو لست متأكدًا:

- لا تخترع.
- لا تنفذ تعديل عشوائي.
- ابحث داخل المشروع.
- اقرأ الملفات.
- اسأل سؤالاً محددًا.
- أو اكتب افتراضك قبل التنفيذ.

صيغة مقبولة:

I found two possible causes. I will fix the smaller and safer one first.

20. أسلوب تنفيذ الموديولز

أي موديول جديد يجب أن يُبنى بالترتيب:

1. Types / contracts
2. Validation schemas
3. Database schema
4. Service layer
5. API controller/routes
6. UI pages
7. Permissions
8. Audit log
9. Tests/build
10. Documentation

لا تبدأ بالواجهة فقط ثم تحاول تركيب backend عشوائيًا.

21. سياسة المهام الصغيرة

كل مهمة يجب ألا تتجاوز غالبًا:

- 1 إلى 5 ملفات.
- تغيير واحد واضح.
- build واحد واضح.
- نتيجة قابلة للمراجعة.

لو المهمة تحتاج أكثر من ذلك، قسمها.

مثال سيئ:

```
Build the whole ERP.
```

مثال جيد:

```
Implement Lead status change API with validation, permission check, activity log, and test build.
```

22. طريقة التعامل مع ELY

ELY يجب أن يبني يحذر شديد:

- لا يستخدم بيانات غير معتمدة.
- لا يجيب من غير مصادر عند الحاجة.
- لا يتخطى صلاحيات المستخدم.
- لا ينفذ Actions خطيرة بدون confirmation.
- كل tool call يتم تسجيله.
- كل feedback يتم حفظه للمراجعة.
- المعرفة لا تصبح approved تلقائيًا.

23. طريقة التعامل مع n8n

n8n لا يعدل DB مباشرة.

القواعد:

```
n8n receives event
n8n performs automation
n8n calls Matrix API
Matrix API validates and writes data
Matrix logs result
```

كل workflow يجب أن يحتوي على:

- eventId
- idempotency
- retry
- error handling
- automation log

24. معيار النهاية Done Definition

لا تعتبر المهمة منتهية إلا إذا تحقق الآتي:

- الكود يعمل.
- `build` نجح.
- `TypeScript errors` لا توجد.
- ظاهرة `runtime` لا توجد أخطاء.
- مكشوفة `secrets` لا توجد.
- الصلاحيات محترمة.
- تم تحديثه عند وجود أخطاء `error-analysis.md`.
- تم تلخيص الملفات المعدلة.

25. رسالة الالتزام للمطور/الـ Agent

قبل تنفيذ أي مهمة، اقرأ هذا الملف.

نفذ أقل تعديل آمن.

لا تعمل أكثر مما يُطلب.

لا تضغط على نفسك بمهام كبيرة.

الجودة والأمان أهم من السرعة.

كل شيء يجب أن يكون قابلاً للفحص، قابلاً للتراجع، وقابلاً للتطوير.